

Whitepaper: Securing Amazon Bedrock AgentCore Agents with Okta

For Questions: [Indranil Jha](#) , [Kundan Kolhe](#) , [Danny Fuhriman](#)

Abstract

Organizations deploying generative-AI agents on **AWS Bedrock AgentCore** frequently require these agents to interact with **Model Context Protocol (MCP) servers** that uphold the same stringent identity and API policies as the broader infrastructure.

This document outlines **two primary integration patterns** that allow for the binding of user identity from Okta to MCP tools behind an Okta authorization server.

1. The Challenge: Securing AI Agent Access to External Resources

When AI agents operating within AWS Bedrock AgentCore need to access external resources or perform actions, a critical security challenge arises: how to ensure these interactions are authenticated, authorized, and attributed to the correct user.

1.1 The Identity Gap

- **AgentCore identity attribution:** Although Agentcore can accept an incoming OIDC token for authenticating to an agent, it calls the outbound MCP tools in the the context of agent's own principal. While it can pass the incoming OIDC token to the downstream resources, it does not comply with a separate scoped token with proper user attribution to a target resource/tool. The end-to-end identity attribution story thus becomes disjointed when combined with a need for preserving least privilege zero trust token propagation in an agentic workflow.
- **AgentCore Runtime limitations:** The AgentCore Runtime invokes your agent with a payload, which may include user prompts and tokens, but it does **not** inherently handle Okta-specific token exchange processes.
- **Protected MCP servers expectations:** Typical MCP servers, especially those mirroring existing APIs (e.g., HR or internal tools), expect requests to include an OAuth based authorization token. This access token must be issued for a defined audience by an OAuth **authorization server** and validated at the resource.
- **id_token vs. access_token:** The user's OIDC **id_token** is issued within the context of the web application's client. It should **not be automatically valid** as an API bearer token for a separate

MCP resource server. This means the **id_token** alone cannot grant access to external, protected MCP resources.

1.2 Okta's Solution: Cross-App Access (XAA) and ID-JAG

Cross-App Access (XAA) in Okta provides a mechanism for a workload (like an AI agent) to exchange a subject token (specifically, the user's **id_token**) for an **ID-JAG token**. This ID-JAG token then allows the flow to **resume** against the **target authorization server** to obtain an **access token** that is correctly scoped for the MCP (or API) layer. This is a two-step federated delegation pattern:

1. **ID-JAG step**: Binds the user's identity to the agent/workload context against the custom authorization server's audience.
2. **Token Exchange step**: Exchanges the ID-JAG token for an **access token** that the MCP server can validate based on its issuer, audience, and required scopes.

1.3 What "Protected MCP" Entails

A **protected MCP server** validates incoming requests using tokens issued by a **custom authorization server**, establishing the same trust boundary one would use for a standard REST API. The agent's role (or a proxy's) is to present this validated token on every MCP request made.

2. Common Building Blocks for Integration

Regardless of the specific pattern used, several key components are involved in securing Bedrock AgentCore agents with Okta:

- **Web application**: This is typically a web application that handles the Okta login for a human user, for example a chat client. It performs the OIDC sign-in with Okta, obtains the **id_token**, and then invokes AgentCore with the user prompt and **id_token** in the invocation payload.
- **AgentCore Runtime**: This AWS service hosts the Strands (or similar) agent and forwards the invocation payload to the agent's endpoint.
- **Strands agent**: This is the AI agent that loads MCP tools, interacts with the model, and executes tool calls over MCP HTTP transport.
- **Okta (Org AS + Custom AS)**: Okta acts as the identity provider, issuing user tokens and, via XAA, generating tokens usable by the MCP resource server.

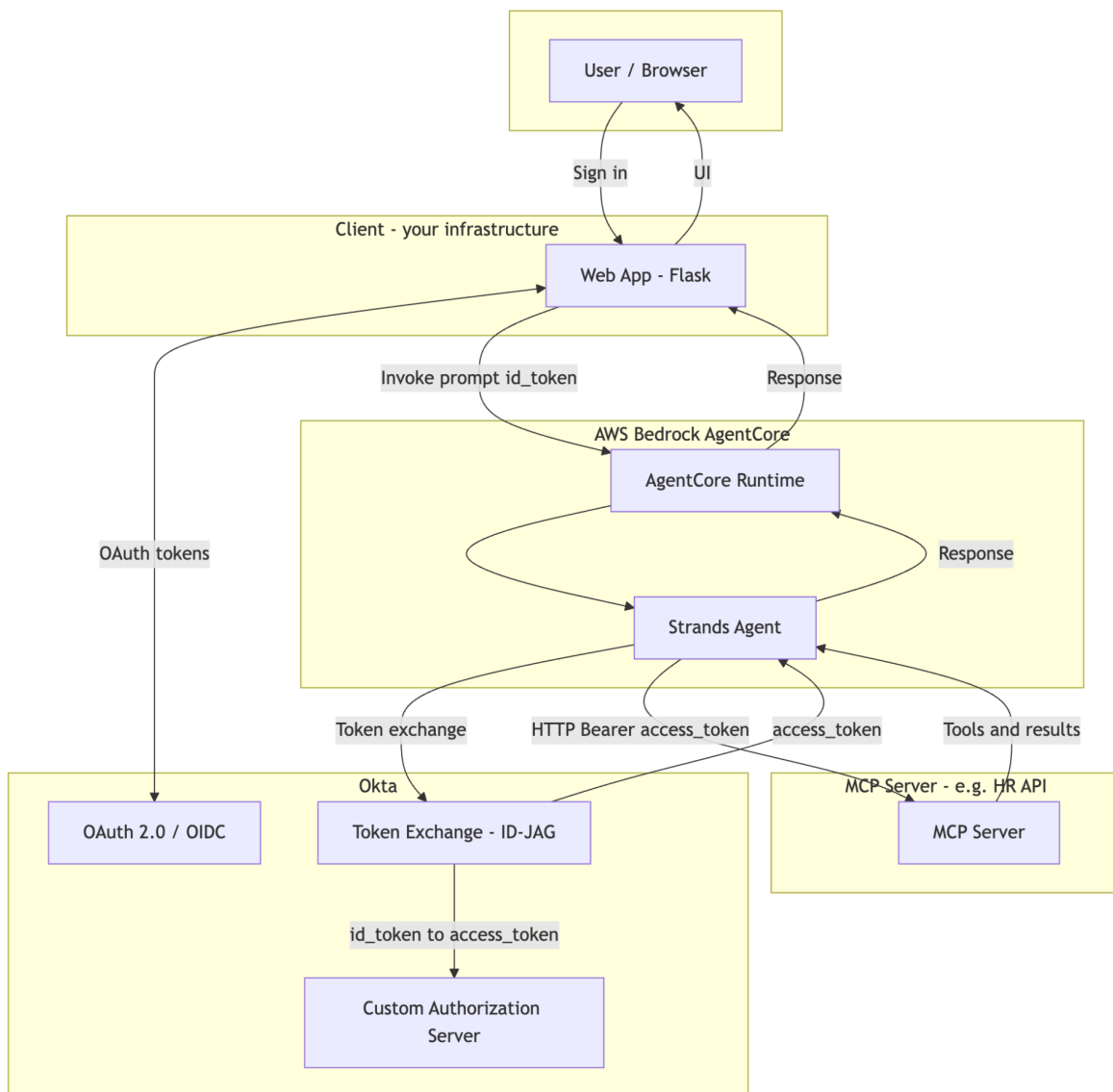
3. Integration Patterns

Here we illustrate two distinct architectural patterns for this integration. The key difference lies in where XAA runs and how the MCP endpoint is reached (direct URL versus Gateway + adapter).

3.1 Pattern A: Direct Agent to XAA-Protected MCP (no Gateway)

Summary:

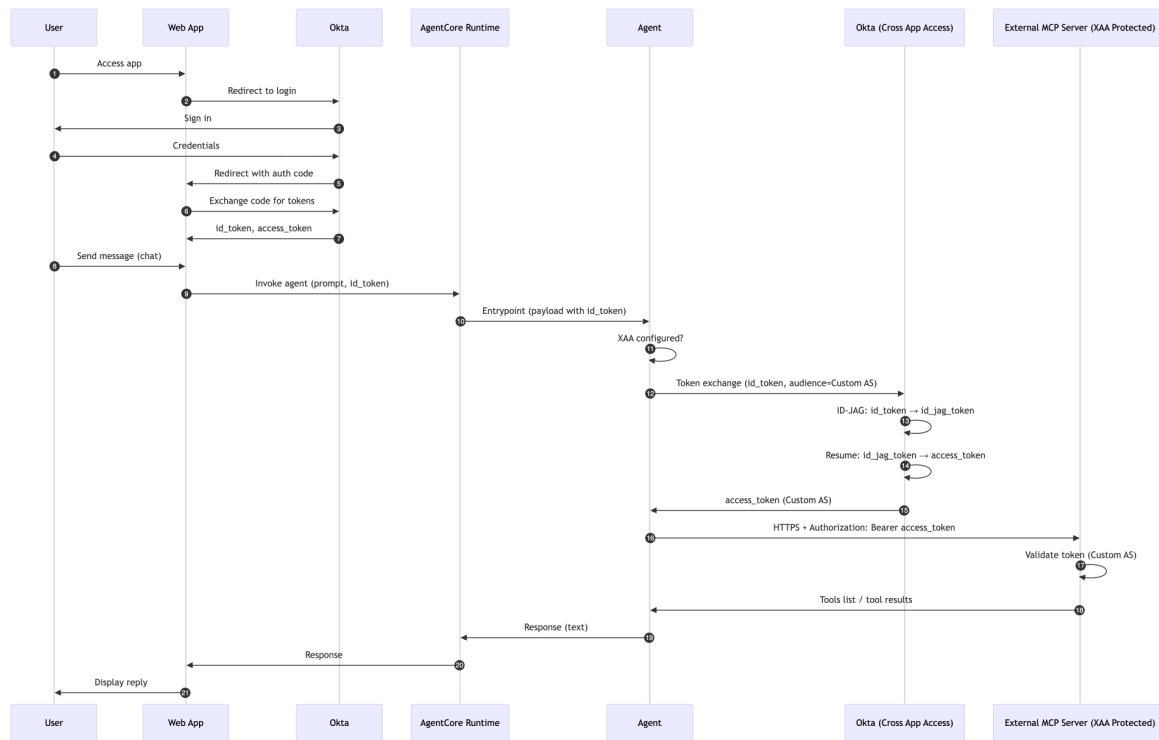
In this pattern, the AI agent itself is responsible for performing the Okta Cross-App Access token exchange. The agent resolves the `id_token` from the invocation payload, uses the Okta Python SDK's `CrossAppAccessFlow` to obtain the custom authorization server's access token, and then uses this token to make calls to the MCP server.



End-to-End Sequence:

1. The user accesses the web application and signs in via Okta SSO.
2. The web application exchanges the authorization code for an id_token from Okta.
3. The user sends a message (e.g., chat) to the application.
4. The application invokes the AgentCore runtime with the user's prompt and id_token.
5. The AgentCore runtime passes the payload, including the id_token, to the Strands agent.
6. The agent, configured for XAA, performs a token exchange with Okta (using id_token and targeting the custom AS).
7. Okta executes the ID-JAG step (id_token to id_jag_token) and then the token exchange step (id_jag_token to access_token from the Custom AS).

8. Okta returns the custom AS access_token to the agent.
9. The agent then makes HTTPS requests to the MCP server, including the Bearer token header.
10. The MCP server validates the token against its custom authorization server.
11. The MCP server returns tools or results to the agent, which then responds through AgentCore and the web app to the user.



Okta SDK Usage:

This pattern relies on the [Okta Python SDK](#)'s CrossAppAccessFlow. The agent configures an OAuth2Client with client assertion (JWT bearer for the agent workload) and defines a CrossAppAccessTarget pointing to the custom authorization server issuer. It then calls flow.start with the user's id_token as the subject token and flow.resume to complete the exchange and receive the access token.

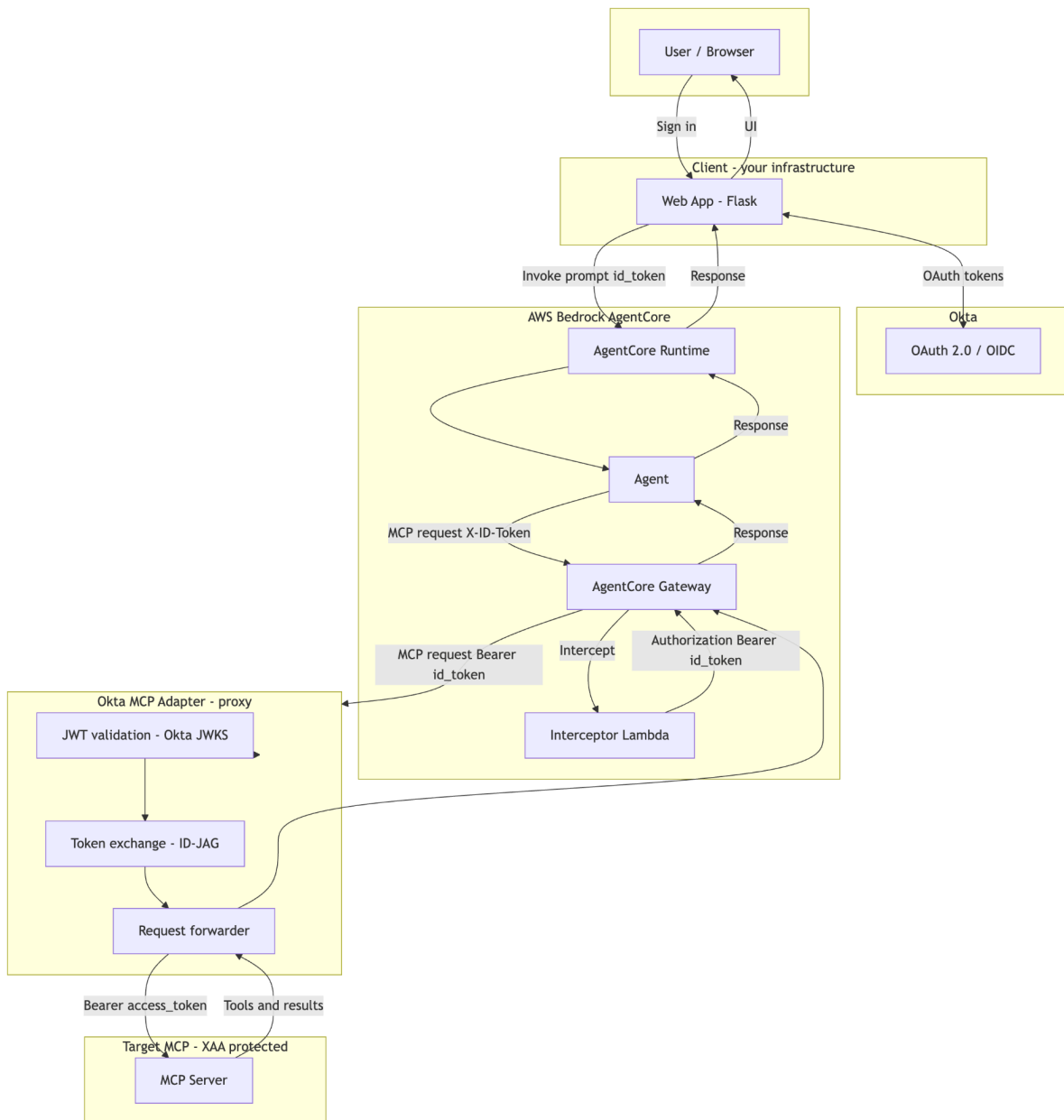
Operational Characteristics:

- **Pros:** Fewer components, as no separate Gateway or Lambda is required. The agent has full control within its container.
- **Cons:** XAA credentials and key material (private keys, client configuration) must be available to the agent's runtime, which necessitates careful secret management (e.g., using AWS Secrets Manager or environment variables).

3.2 Pattern B: AgentCore Gateway, Interceptor, and Okta MCP Adapter

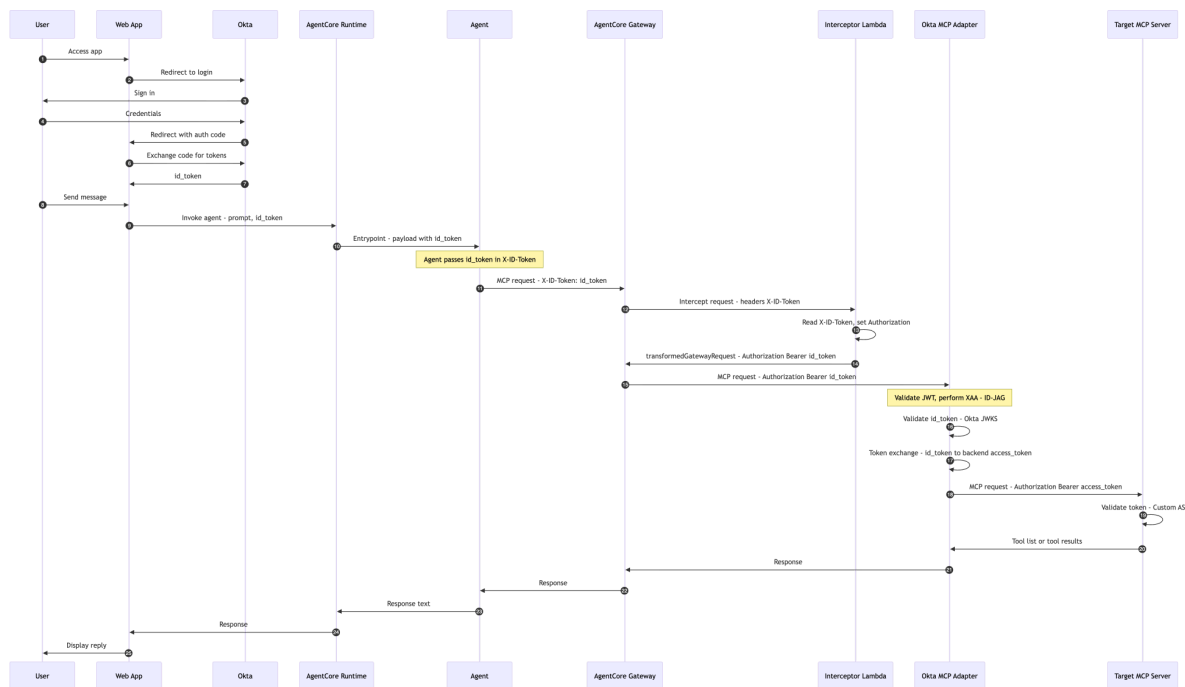
Summary:

This pattern centralizes JWT validation and XAA in a dedicated Okta MCP Adapter that acts as a proxy. The agent sends the user's `id_token` in an allowlisted HTTP header (e.g., `X-ID-Token`). An AgentCore Gateway with a Lambda interceptor transforms this into an authorization bearer token header before routing to the Okta MCP Adapter. The adapter then performs the XAA and forwards the request with the correct token to the target MCP server.



End-to-End Sequence:

1. The user accesses the web application and signs in via Okta SSO, obtaining an `id_token`.
2. The application invokes the AgentCore runtime with the user's prompt and `id_token`.
3. The Strands agent makes an MCP request to the AgentCore Gateway, passing the `id_token` in a custom header like `X-ID-Token`.
4. The Gateway's Lambda interceptor intercepts this request, reads the `X-ID-Token`, and transforms it into an `Authorization: Bearer <id_token>` header for the next hop.
5. The Gateway routes the request to the Okta MCP Adapter.
6. The Okta MCP Adapter validates the `id_token` (using Okta JWKS) and then performs the XAA (ID-JAG token exchange) to obtain an access token for the backend MCP server.
7. The adapter forwards the MCP request to the target MCP server, now with the `Authorization: Bearer <access_token>` header.
8. The MCP server validates the token against its custom authorization server and returns results to the adapter.
9. The adapter forwards the response back through the Gateway and AgentCore runtime to the agent and then to the web app for display to the user.



Agent-Side Contract:

In this pattern, the agent does not implement XAA. Instead, its HTTP client must send the `id_token` in an allowlisted header (e.g., `X-ID-Token`) so the Lambda interceptor can rebuild the `Authorization` header.

Operational Characteristics:

- **Pros:** Centralizes JWT validation and XAA logic within the Okta MCP Adapter, which can be managed separately. The agent's image remains free of XAA credentials and key material, shifting this responsibility to the adapter. The Gateway can provide a single MCP entry URL and enforce optional policies.
- **Cons:** This pattern introduces more components (Gateway, Lambda, adapter deployment), increasing architectural complexity. Careful alignment of Okta redirect URIs, Gateway authorizer settings, Lambda permissions, and adapter configuration is crucial.

4. Comparison of Integration Patterns

Criterion	Pattern A — Direct MCP + in-agent XAA	Pattern B — Gateway + interceptor + Okta MCP Adapter
MCP network path	Agent → MCP HTTPS	Agent → Gateway → Lambda → Adapter → MCP
Where XAA runs	Inside AgentCore agent	Inside Okta MCP Adapter (proxy)
Bearer token presented	Custom AS access_token from agent	Custom AS access_token from adapter
Okta SDK in agent	Yes	No
Gateway / Lambda	Not used	Required for header injection and routing
Typical use	Minimal AWS footprint; own secrets in runtime	Centralize identity and token policy at the adapter

5. Conclusion

Amazon Bedrock AgentCore can be effectively integrated with **Okta**-protected MCP servers by explicitly modeling the distinction between **user OIDC tokens** and **resource access tokens**.

Cross-App Access (ID-JAG) is the key Okta capability that bridges this gap. The choice between implementing **direct in-agent XAA** (Pattern A) or utilizing a **Gateway + Okta MCP Adapter** (Pattern B) primarily involves a trade-off between **operational centralization** (offered by the adapter) and **deployment simplicity** (of the direct approach). Fundamentally, both patterns leverage Okta's robust trust model at the MCP resource server to ensure secure, attributed, and policy-driven interactions for AI agents.